

App Dev Project Report

1. Student Details

Name: Priyadarshini Palanirajan
Roll Number: 24f2004973
Email: 24f2004973@ds.study.iitm.ac.in

About Me:

I am a diploma level data science student with a strong interest in software development, databases, and web application development. I enjoy building practical applications that solve real-world problems and help improve user experiences through technology.

2. Project Details

Project Title: Trekking Management Application

Problem Statement

To design and develop a web-based trekking management system that enables admin, trek staff, and trekkers to efficiently manage trekking events, registrations, participant records, and trekking history through role-based access control.

Approach:

The application was built using Flask and SQLite with a modular architecture. It provides separate dashboards for administrators, trek staff, and trekkers, allowing secure management of treks, bookings, participant records, and trekking history.

3. AI / LLM Declaration

I used **ChatGPT (GPT-5)** to assist with debugging Flask routes, improving SQLAlchemy model design, refining validation logic, and code structure suggestions. The extent of AI/LLM usage is approximately **20–25%**, primarily for debugging assistance. All implementation, testing and integration were completed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-Login	User authentication and session management
Flask-WTF	Form handling and validation
WTForms	Form generation and backend validation
SQLAlchemy	ORM for database management
SQLite	Relational database storage
Jinja2	Dynamic HTML template rendering
Bootstrap 4	Responsive frontend design
Flask-Bcrypt	Password hashing and security
HTML5	Frontend page structure

5. Database Schema / ER Diagram

Tables:

1. **User** - Stores information about all system users (Administrator, Staff, Trekker).
2. **Staff** - Stores additional information specific to staff members.
3. **Trek** - Stores trek information and assigned staff.
4. **Booking** - Stores trek registrations by trekkers.
5. **TrekHistory** - Stores completion records, ratings, and reviews.

Relationships:

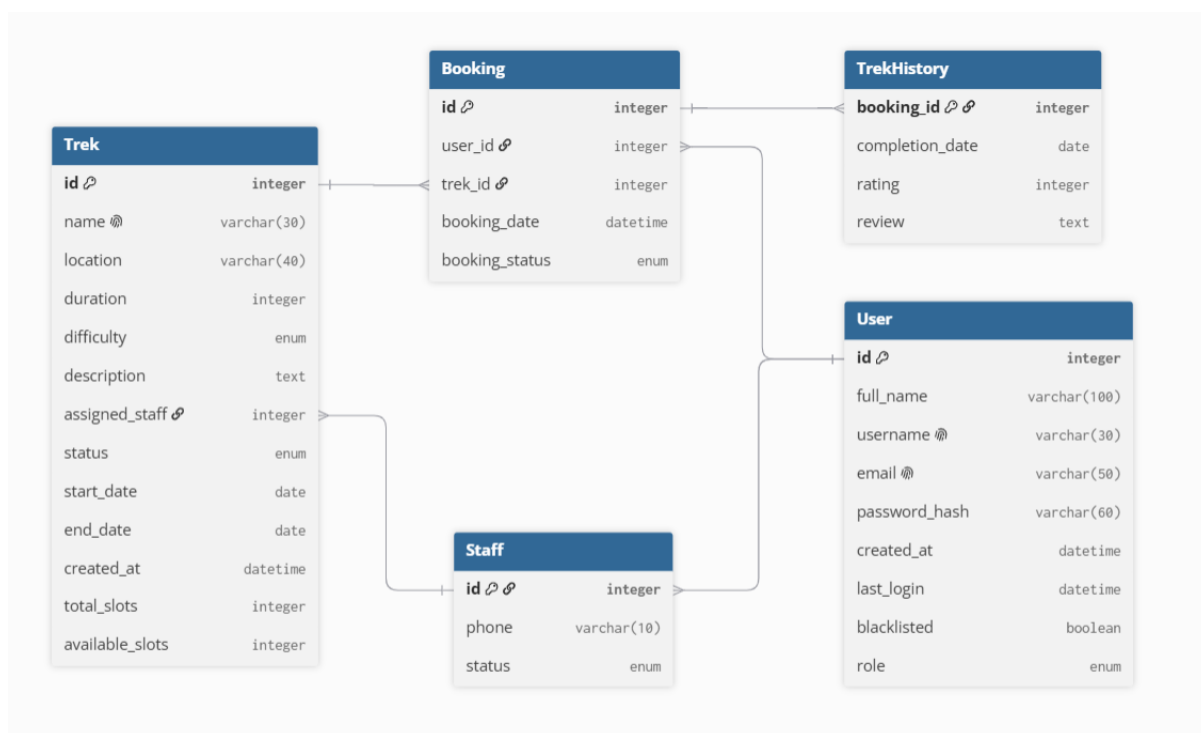
One user can make multiple bookings: User (1) ----- (M) Booking

One trek can have multiple bookings: Trek (1) ----- (M) Booking

One booking can have one trek history record: Booking (1) ----- (1) TrekHistory

A staff member is a specialized user: User (1) ----- (0..1) Staff

One staff member can manage multiple treks: Staff (1) ----- (M) Trek



(created using dbdiagram.io)

6. API Design

No REST APIs were implemented as the project primarily focuses on server-side rendered web pages using Flask and Jinja2. Application functionality is handled through Flask route controllers that process requests, interact with the database, and render dynamic templates.

7. Architecture and Features

Architecture Overview:

The project follows a modular Flask architecture:

File / Folder	Purpose
run.py	Application entry point used to start the Flask server
trekking/__init__.py	Initializes Flask app, database, Flask-Login, Bcrypt, and application configuration
trekking/models.py	Contains SQLAlchemy database models and relationships
trekking/forms.py	Contains WTForms definitions and validation rules
trekking/routes.py	Contains all Flask routes, controllers, and business logic
trekking/templates/	Jinja2 HTML templates for Admin, Staff, and Trekker interfaces

Features Implemented:

- **Models (models.py)** manage database entities such as Users, Staff, Treks, Bookings, and Trek History.
- **Views (templates/)** contain Jinja2 templates rendered dynamically based on user roles and database data.
- **Controllers (routes.py)** handle requests, perform validation, interact with database models, and render appropriate templates.
- **Forms (forms.py)** centralize user input validation using Flask-WTF and WTForms.
- Authentication and authorization are implemented using **Flask-Login**, ensuring role-based access control for Administrators, Trek Staff, and Trekkers.

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1kMJKzrfau8xUXAnqLM2d_7wMUgc7KktD/view?usp=sharing

(Accessible to all with "View" permission)